Registro de clientes en espera

Informe técnico y funcional del punto 2 del trabajo práctico final.

1. Objetivo y alcance

Gestionar a los clientes que solicitan un producto sin disponibilidad mediante el **TDA Cola de la cátedra**. Al reponer stock se simula la notificación al primer cliente y se lo retira de la espera, respetando el orden de llegada.

El módulo está conectado con el inventario implementado mediante lista enlazada, una base SQLite, el servidor HTTP nativo y la interfaz web. El registro utiliza clientes existentes; el alta de nuevos clientes no forma parte de esta funcionalidad.

2. Reglas adoptadas

Regla	Comportamiento definido
Existencia	El cliente y el producto deben estar registrados.
Disponibilidad	Solo se admiten nuevas esperas cuando el stock es cero.
Duplicados	Una sola espera activa por cliente y producto. Un cliente puede esperar varios productos.
Orden FIFO	Cada producto tiene una cola independiente: el primero que ingresa es el primero que se atiende.
Reposición	El cambio de stock de cero a positivo atiende a un solo cliente, si hay alguien en espera.
Notificación	Se genera un aviso simulado en pantalla. No se envía un correo, no se reserva ni se descuenta stock y no se registra una compra.
Persistencia	Las esperas se guardan en SQLite y las colas se reconstruyen al iniciar el servidor.
Errores	Si SQLite rechaza una operación, no se modifica la cola. La reposición y la eliminación de la espera se confirman juntas.

Referencia de la consigna

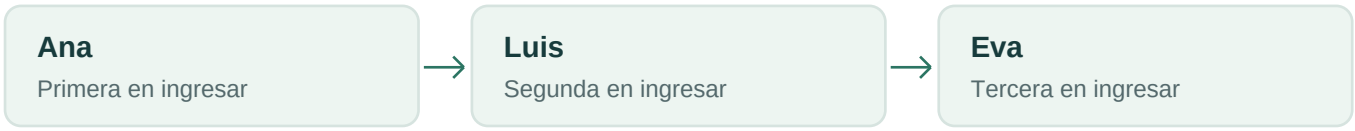
Fuente: 02.TP-Final.TIENDA-III.2026.pdf, página 1, apartado "Registro de clientes en espera (TDA Cola)". Las reglas de duplicados, stock cero y una cola por producto concretan el funcionamiento acordado para el proyecto.

TDA Cola y organización

3. Representación de las esperas

Un diccionario relaciona cada **id_producto** con una instancia de **Cola**. El diccionario localiza la estructura; los registros se almacenan dentro del TDA. Cada registro contiene id_espera, id_cliente, id_producto y fecha_espera.

Producto de ejemplo: teclado sin stock



Sale por el frente **Los nuevos registros ingresan al final**

Ejemplo ilustrativo: aunque se repongan varias unidades, se atiende a Ana y la cola restante queda Luis, Eva. Una nueva llegada se incorpora después de Eva.

4. Primitivas utilizadas

Primitiva	Uso en el módulo
Cola / arribo	Crear la cola e incorporar una espera al final.
cola_vacia / en_frente	Comprobar si hay registros y consultar al primero sin retirarlo.
atencion	Retirar al primer registro después de confirmar la operación en SQLite.
tamanio / mover_al_final	Consultar todos los registros mediante una vuelta completa, conservando el orden final.

Las operaciones básicas arribo, atencion y en_frente son O(1). Consultar una cola con n registros es O(n) y genera una lista de copias para responder a la interfaz. El recorrido rota temporalmente la cola; por eso también se ejecuta bajo el bloqueo compartido. La lógica no manipula directamente frente, final ni sig.

5. Responsabilidades por archivo

Archivo del proyecto	Responsabilidad
BACKEND/tad_cola.py	Implementación de la estructura y sus primitivas.
BACKEND/esperas.py	Carga, consulta, registro, preparación del aviso y retiro de la primera espera.
BD/persistencia.py	Consultas SQL, validaciones del registro y guardado transaccional.
BACKEND/inventario.py	Detección de la reposición y coordinación entre lista y cola.
BACKEND/servidor.py	Rutas HTTP y bloqueo compartido entre solicitudes.
FRONTEND/index.html, app.js y estilos.css	Controles, comunicación con la API, mensajes y presentación adaptable.

Persistencia y flujo de datos

6. Almacenamiento en SQLite

La tabla **esperas** pertenece a BD/tablas.db. Sus claves foráneas relacionan clientes y productos; la conexión activa PRAGMA foreign_keys = ON.

Campo	Tipo y restricción	Finalidad
id_espera	INTEGER, PK, AUTOINCREMENT	Identificar el registro y establecer el orden de llegada.
id_cliente	INTEGER, NOT NULL, FK	Referenciar al cliente registrado.
id_producto	INTEGER, NOT NULL, FK	Determinar la cola correspondiente.
fecha_espera	TEXT, NOT NULL	Guardar el ingreso en formato ISO 8601 con zona UTC.

El orden de reconstrucción es ORDER BY id_espera. La fecha permite mostrar el ingreso, pero no determina la prioridad. La prohibición de duplicados se comprueba dentro de una transacción BEGIN IMMEDIATE; el esquema actual no tiene una restricción UNIQUE para el par cliente-producto.

7. Registro de una nueva espera

1. La pantalla envía los identificadores del cliente y del producto. El servidor valida el JSON y los IDs.

2. Bajo el bloqueo compartido, registrar_espera genera la fecha UTC y solicita el guardado.

3. SQLite comprueba existencia, stock cero y ausencia de duplicados dentro de la misma transacción.

4. Confirmado el guardado, arribo incorpora el registro a la cola. La API devuelve el registro creado.

8. Reposición y atención

1. El inventario compara el stock anterior con el nuevo. Solo el paso de cero a positivo activa la atención.

2. Se consulta al primero y se prepara el mensaje sin retirar todavía la espera.

3. SQLite verifica que sea la primera espera del producto y guarda el nuevo stock junto con la eliminación del registro. Si falla, revierte ambos cambios.

4. Después de confirmar SQLite, se actualiza la lista y atencion retira al primero. La respuesta incluye el aviso simulado.

Un único Lock del servidor coordina las solicitudes de inventario y colas. Al iniciar se reconstruyen las colas desde SQLite, sin depender de la memoria de la ejecución anterior. Las consultas de espera devuelven copias de los registros.

API, interfaz y verificación

9. Rutas disponibles

Método y ruta	Resultado
GET /api/clientes	Clientes registrados para el selector.
GET /api/productos/ID/esperas	Cola FIFO del producto; [] si está vacía; 404 si no existe el producto.
POST /api/esperas	Recibe id_cliente e id_producto. Devuelve 201 y la espera creada.
PUT /api/productos/ID	Modifica el inventario y devuelve notificacion cuando atiende una espera por reposición.

Ejemplo de cuerpo para POST /api/esperas: `{"id_cliente": 1, "id_producto": 3}`. La fecha y el ID de espera se generan en el backend. Las solicitudes inválidas o que incumplen las reglas reciben 400; un formato distinto de JSON, 415; los errores SQLite controlados, 500.

10. Uso desde la pantalla

Desde el catálogo se puede elegir "Anotar cliente" si no hay stock o "Ver espera" si hay disponibilidad. La sección "Clientes en espera" permite seleccionar el producto y el cliente, consultar el orden y actualizar los datos. El primer cliente aparece destacado; quienes ya esperan el producto están deshabilitados en el selector.

Al reponer desde la edición del producto, se muestra el aviso "Cliente notificado" y se actualiza la cola. Los estados de carga, ausencia de clientes, cola vacía y errores tienen mensajes específicos. Las respuestas tardías de otro producto se descartan y los nombres se insertan como texto. La interfaz usa HTML, CSS y JavaScript nativos.

11. Pruebas realizadas durante la implementación

Comprobación	Resultado observado
Registro y validaciones	Guardado correcto; rechazo de duplicados, IDs inválidos, registros inexistentes y productos con stock.
FIFO e independencia	Orden conservado al consultar; colas separadas por producto; un cliente atendido por reposición.
Concurrencia y fallos	Dos registros simultáneos del mismo par producen una sola espera. Un fallo al eliminar revierte la reposición.
Persistencia y HTTP	Recarga de colas correcta y respuestas verificadas mediante solicitudes HTTP reales.
Interfaz en navegador	Registro, aviso, actualización, errores y reintentos comprobados en escritorio y móvil, sin errores JavaScript.

Las comprobaciones usaron bases temporales; no alteraron los registros reales. La prueba de interfaz está en tmp/pruebas-esperas/servidor_prueba.py y su automatización en interfaz.cjs. Este informe documenta las verificaciones previas, no una nueva ejecución de esas pruebas.

Estado: punto 2 implementado e integrado. La pila de transacciones y el árbol de popularidad corresponden a los puntos siguientes; actualmente la espera no genera compras automáticas.